สมาชิกและหน้าที่ที่รับผิดชอบ

ชื่อ	รหัสนักศึกษา	หน้าที่ที่รับผิดชอบ
นางสาวกมลพร เกตุแก้ว	673380571-1	IaC/DevOps
นางสาวพรีมภัทร ภาวัฒนวคุณ	673380594-9	IaC/DevOps
นางสาวมุกดา บุญประจันทร์	673380598-1	QA/Test
นางสาวสรนันท์ บุสดี	673380605-0	Network Architects

Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

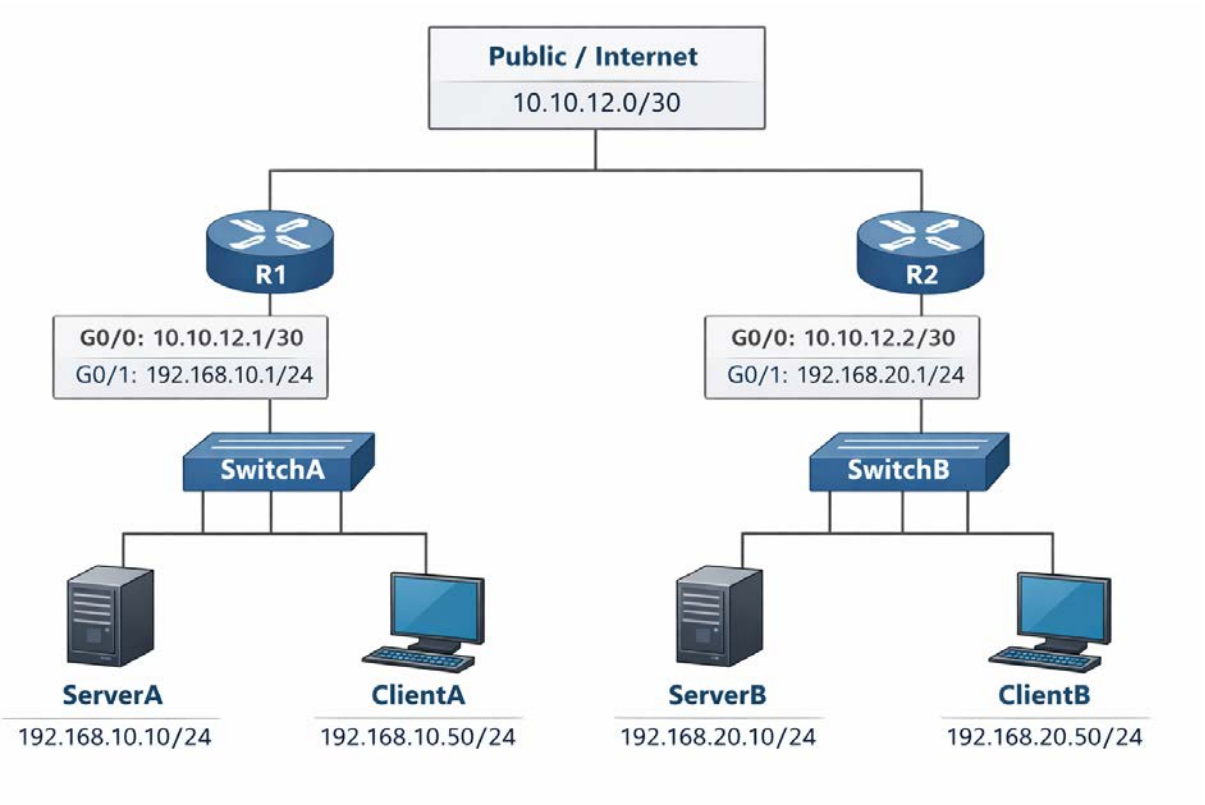
👥 TEAM BREAKDOWN (5–6 students)

Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```
□                10.10.12.0/30 "Public/Internet"
      ┌──────────────────────────┐
      |            |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
      |            |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
      |            |
         SwitchA                     SwitchB
      |            |
 ServerA (mockup)           ServerB (mockup)
 ClientA                        ClientB
□
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

🧱 R2 CONFIGURATION (LAN B)

```
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
```

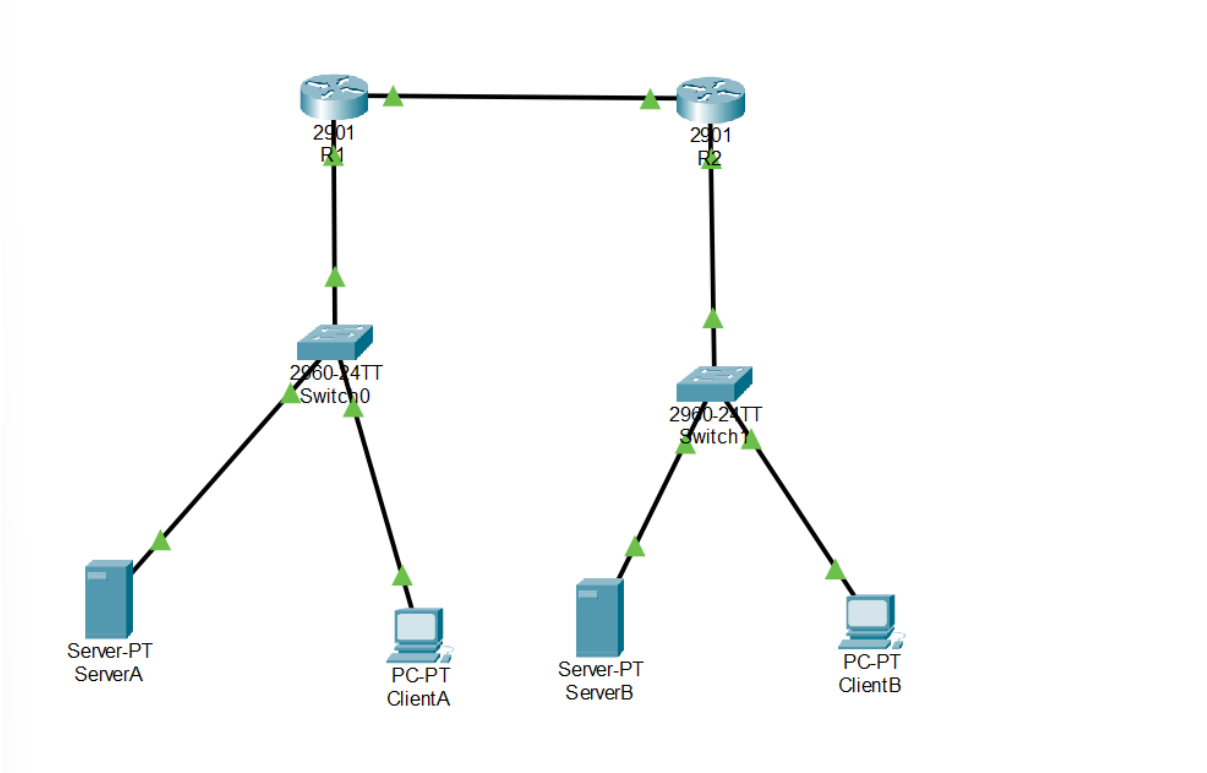
```
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

Result :



🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
show ip route
show ip nat translations
```

R1 :

```
R1>enable
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is 10.10.12.2 to network 0.0.0.0

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2
S*   0.0.0.0/0 [1/0] via 10.10.12.2

R1#show ip nat translations
R1#
```

R2 :

```
R2#enable
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is 10.10.12.1 to network 0.0.0.0

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.2/32 is directly connected, GigabitEthernet0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/1
L       192.168.20.1/32 is directly connected, GigabitEthernet0/1
S*   0.0.0.0/0 [1/0] via 10.10.12.1

R2#show ip nat translations
R2#
```

From ClientA:

ping 192.168.20.10 ←Remote server reachable

Result :

```
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Request timed out.
Reply from 192.168.20.10: bytes=32 time=1ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>
```

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠 CLONE REPO

```
git clone https://github.com/boonsup/automation.git
cd automation
```

🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
ServerA.env U ✕   ServerB.env U   manage.py
mockup-infra > ServerA.env
1   PUBLIC_NET_SUBNET=192.168.10.0/24
2   HOST_IP=192.168.10.10
3   REMOTE_NET_PUBLIC=10.10.12.0/30
```

□ServerB (`ServerB.env`)

```
ServerA.env U   ServerB.env U ✕   manage.py
mockup-infra > ServerB.env
1   PUBLIC_NET_SUBNET=192.168.20.0/24
2   HOST_IP=192.168.20.10
3   REMOTE_NET_PUBLIC=10.10.12.0/30
```

🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
D:\network\automation\mockup-infra>python manage.py init

🔧 Initializing Mockup Infrastructure...
==================================================

📅 [Layer 5/6 - Session] Generating TLS Certificates...
D:\network\automation\mockup-infra\manage.py:118: DeprecationWarning: datetime.dat
  .not_valid_after(datetime.utcnow() + timedelta(days=365))
✅ Certificates generated successfully:
   - Private Key: D:\network\automation\mockup-infra\certs\server.key
   - Certificate: D:\network\automation\mockup-infra\certs\server.crt
   - Common Name: api.mockup.test
   - Subject Alt Names: localhost, api.mockup.test, 127.0.0.1, 172.18.0.1, 172.19.

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
The system cannot find the path specified.
'{}'
⚠️ Private network not found or not isolated. Creating networks...
🔧 Executing: podman network create public_net 2>/dev/null || true
The system cannot find the path specified.
'true' is not recognized as an internal or external command,
operable program or batch file.
🔧 Executing: podman network create --internal private_net 2>/dev/null || true
The system cannot find the path specified.
'true' is not recognized as an internal or external command,
operable program or batch file.
✅ Networks created successfully

✅ Infrastructure initialized successfully

D:\network\automation\mockup-infra>
```

```
(venv) nitro@Preemphat-P:~/automation/mockup-infra$ python manage.py deploy

🚀 Deploying Mockup Infrastructure Stack
   Bare Metal → L3 → L4 → L5/6 → L7
==================================================

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
[
    {
        "name": "private_net",
        "id": "dfc64df5f8a67c4adbba4e7f2bfc3abcf1b43eee55ea3ad776a1c61a19b58b7e",
        "driver": "bridge",
        "network_interface": "podman2",
        "created": "2026-03-26T23:44:06.742325779+07:00",
        "subnets": [
            {
                "subnet": "10.89.1.0/24",
                "gateway": "10.89.1.1"
            }
        ],
        "ipv6_enabled": false,
        "internal": true,
        "dns_enabled": true,
        "ipam_options": {
            "driver": "host-local"
        }
    }
]
✅ Private network is isolated (internal: true)
🔧 Executing: podman-compose up -d --remove-orphans
ff358ab7433bc3a4b60916df91fc3d57d7c64a2715bbcf094e03f152530af340
Error: unable to start container "ff358ab7433bc3a4b60916df91fc3d57d7c64a2715bbcf094e03f152530af340":
rt 443, you can add 'net.ipv4.ip_unprivileged_port_start=443' to /etc/sysctl.conf (currently 1024),
 listen tcp 0.0.0.0:443: bind: permission denied
(venv) nitro@Preemphat-P:~/automation/mockup-infra$
```

🟡 VERIFY CONTAINERS

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

podman ps

ผลลัพธ์พบว่า container ที่จำเป็นทำงานอยู่เรียบร้อย ได้แก่

stateless-api ทำงานอยู่ (port 3000)

stateful-api ทำงานอยู่ (port 3001)

```
(venv) nitro@Preemphat-P:/mnt/d/network/automation/mockup-infra$ podman ps
CONTAINER ID  IMAGE                                       COMMAND               CREATED        STATUS             PORTS                                       NAMES
4af2cbdbbe0d  localhost/mockup-infra_public_app:latest    python app.py         3 minutes ago  Up About a minute                                              mockup-public-web
995b1665666e  localhost/mockup-infra_intranet_api:latest  python api.py         3 minutes ago  Up About a minute                                              mockup-intranet-api
4f04a9a06397  docker.io/library/nginx:alpine              nginx -g daemon o...  3 minutes ago  Up About a minute  0.0.0.0:8080->80/tcp, 0.0.0.0:8443->443/tcp  mockup-gateway
d0a8ab9dc7a3  localhost/stateless-api:v2                  node start-statel...  3 minutes ago  Up 3 minutes       0.0.0.0:3000->3000/tcp                      stateless-api
a5415f41de2d  localhost/stateful-api:v2.1                 node start-statef...  3 minutes ago  Up 3 minutes       0.0.0.0:3001->3001/tcp                      stateful-api
```

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```
(venv) nitro@Preemphat-P:~/automation/mockup-infra$ curl http://localhost:3000/info
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1
163Z","requestCount":1,"clientId":"anonymous","randomValue":0.1658691756262476,"message":"I have no mem
t is independent.","note":"This is a stateless server - I don't remember you from previous requests."},
```

→ Stateless

From ClientA:

```
curl -X POST http://192.168.10.10:3001/session
```

```
(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-stateless-stateful/phase1-mockup$ curl -X POST http://localhost:3001/session \
-H "Content-Type: application/json" \
-d '{"userId":"user001"}'
{"status":"success","message":"Session created successfully","data":{"session":{"id":"bd1eb0be-4928-4402-a88d-2215983a167f","userId":"user001","created
At":"2026-03-27T11:36:18.193Z","expiresAt":"2026-03-27T11:51:18.193Z"},"user":{"id":"user001","name":"Test User 1","email":"user1@example.com"},"note":
"Session created. Include Session-ID header in subsequent requests to maintain state."},"timestamp":"2026-03-27T11:36:18.194Z"}(venv) nitro@Preemphat-P
:/mnt/d/network/automation/week02-stateless-stateful/phase1-mockup$
```

Store the returned `Session-ID`, then:

```
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

```
(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-stateless-stateful/phase1-mockup$ curl http://localhost:3001/session \
-H "Session-ID: bd1eb0be-4928-4402-a88d-2215983a167f"
{"status":"success","message":"Session information","data":{"session":{"id":"bd1eb0be-4928-4402-a88d-2215983a167f","userId":"user001","createdAt":"2
-03-27T11:36:18.193Z","lastAccessed":"2026-03-27T11:42:08.299Z","expiresAt":"2026-03-27T11:51:18.193Z","data":{}},"note":"Session retrieved. This se
r remembers your session across requests."},"timestamp":"2026-03-27T11:42:08.300Z"}(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-statele
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
curl -X POST http://10.10.12.2:3001/session
```

```
(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-stateless-stateful/phase1-mockup$ curl http://loca
000/info
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","
mp":"2026-03-27T11:44:44.271Z","requestCount":6,"clientId":"anonymous","randomValue":0.21790861039820375,"me
"I have no memory of previous requests. Each request is independent.","note":"This is a stateless server - I
remember you from previous requests."},"timestamp":"2026-03-27T11:44:44.271Z"}(venv) nitro@Preemphat-P:/mnt/
```

→ ServerB returns a session ID

Then:
Session ID ที่ได้ af77df93-010d-442f-b20c-05e4853221ad

```
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

```
(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-stateless-stateful/phase1-mockup$ curl -X POST http://lo
calhost:3001/session \
-H "Content-Type: application/json" \
-d '{"userId":"user001"}'
{"status":"success","message":"Session created successfully","data":{"session":{"id":"af77df93-010d-442f-b20c-05e4
853221ad","userId":"user001","createdAt":"2026-03-27T11:45:32.272Z","expiresAt":"2026-03-27T12:00:32.272Z"},"user
:{"id":"user001","name":"Test User 1","email":"user1@example.com"},"note":"Session created. Include Session-ID hea
der in subsequent requests to maintain state."},"timestamp":"2026-03-27T11:45:32.272Z"}(venv) nitro@Preemphat-P:/
```

```
(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-stateless-stateful/phase1-mockup$ curl http://localhost:3001/session \
-H "Session-ID: af77df93-010d-442f-b20c-05e4853221ad"
{"status":"success","message":"Session information","data":{"session":{"id":"af77df93-010d-442f-b20c-05e4853221ad","userId":"user001","createdAt":"2026-03-27T11
32.272Z","lastAccessed":"2026-03-27T11:47:10.917Z","expiresAt":"2026-03-27T12:00:32.272Z","data":{}},"note":"Session retrieved. This server remembers your sess
cross requests."},"timestamp":"2026-03-27T11:47:10.917Z"}(venv) nitro@Preemphat-P:/mnt(venv) nitro@Preemphat-P:/mnt/d/network/automation/week02-stateless-state
```

Check session persists across NAT

ในการทดสอบผ่าน simulated internet พบว่า stateful service สามารถรักษา session ได้แม้ request จะผ่าน NAT โดยใช้ Session-ID เดิมในการเรียกใช้งานหลายครั้งและพบว่า server ยังจำสถานะได้อย่างต่อเนื่อง แสดงให้เห็นว่า session persistence ไม่ได้รับผลกระทบจาก NAT เนื่องจาก NAT ทำงานในระดับ network layer ส่วน session ถูกจัดการในระดับ application layer

---

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

จากการทดลองใน lab นี้ทำให้เข้าใจความแตกต่างระหว่าง stateless และ stateful services อย่างชัดเจน โดย stateless service ไม่เก็บข้อมูลของ request ก่อนหน้า ทำให้สามารถ scale ได้ง่ายและเหมาะกับระบบ API ทั่วไป ในขณะที่ stateful

service มีการจัดการ session และสามารถจดจำผู้ใช้งานได้ ซึ่งเหมาะกับระบบที่ต้องมีการรักษาสถานะ เช่น ระบบ login หรือ shopping cart

นอกจากนี้ยังพบว่า NAT และ routing ไม่มีผลต่อ session persistence เนื่องจาก NAT ทำงานในระดับ network layer ในขณะที่ session ถูกจัดการในระดับ application layer ผ่าน Session-ID ดังนั้นแม้ IP address จะถูกแปลงระหว่างทาง session ก็ยังสามารถใช้งานได้ตามปกติ

โดยรวมแล้ว lab นี้ช่วยให้เข้าใจการออกแบบระบบเครือข่ายและ application ร่วมกัน ทั้งในด้าน scalability และ state management ซึ่งเป็นแนวคิดสำคัญในการพัฒนา system ในโลกจริง